

Finite-Element Analysis of a Two-Span Euler–Bernoulli Beam

ME3180 Finite Element Method Theory – Group 7

Abstract

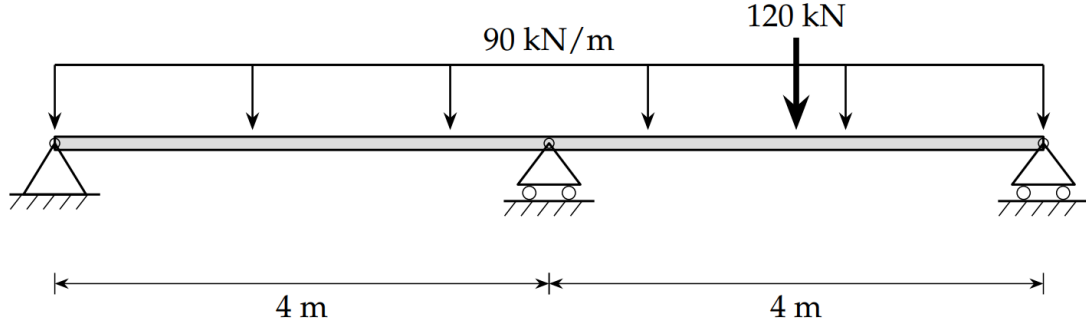
This report develops and verifies a modular finite-element solver for the assigned two-span Euler–Bernoulli beam. The implementation supports spatially varying properties and loads, concentrated forces and moments, internal moment and shear releases, arbitrary-order Hermite interpolation, Gauss–Legendre quadrature, automatic mesh generation, and both h - and p -refinement. The maximum computed deflection is 11.348 mm at $x = 6.180$ m.

1 Problem definition, inputs, and assumptions

The assigned beam has two 4 m spans and simple supports at $x = 0, 4, 8$ m. Its properties and loading are

$$E = 210 \text{ GPa}, \quad I = 100 \times 10^{-6} \text{ m}^4, \quad q = 90 \text{ kN/m}, \quad P = 120 \text{ kN}.$$

The point-load coordinate is not dimensioned in the supplied drawing; it is therefore assumed to act at the midpoint of the second span, $x_P = 6$ m. The adopted convention takes transverse displacement v , distributed load q , and point force P as positive downward. Rotation is $\theta = dv/dx$ (clockwise positive on the plotted beam), while bending moment is positive in sagging. With these choices, positive shear satisfies $V = dM/dx$. Small deflection, linear elasticity, and Euler–Bernoulli kinematics are assumed.



Given: $E = 210 \text{ GPa}$, $I = 100 \times 10^{-6} \text{ m}^4$.

Figure 1: Group 7 beam model.

All problem-specific data are isolated in `input.jl`. The input functions define the beam length, $E(x)$, $I(x)$, element count, nodes per element, restrained DOFs and prescribed values, $q(x)$, point forces, point moments, and internal releases. Thus another compatible beam can be analysed without editing the solver modules. Material quantities are expressed in the consistent kN, m system.

2 Governing differential equation

For an Euler–Bernoulli beam,

$$\kappa = v'', \quad M = -EIv'', \quad V = M'.$$

On the interior of each element $\Omega_e = (x_e, x_{e+1})$, transverse equilibrium gives

$$\boxed{\frac{d^2}{dx^2} \left(EI \frac{d^2 v}{dx^2} \right) = q(x), \quad x \in \Omega_e.} \quad (1)$$

For constant EI this reduces to $EIv^{(4)} = q$. Concentrated forces and moments are placed at mesh nodes by the preprocessor and are added directly to the global nodal load vector; they are therefore not included in the element-domain differential equation.

The essential support conditions are

$$v(0) = v(4) = v(8) = 0, \quad (2)$$

while rotation is free at all three simple supports. The natural end conditions are $M(0) = M(8) = 0$. Across an ordinary element boundary, v , v' , M , and V are continuous. At a support or point force, shear may jump; at a point moment, bending moment may jump. These interface effects are enforced by assembly and nodal loading.

3 Weak form

The fourth-order strong equation would demand four classical derivatives of v . The weak form transfers two derivatives from v to a test function, reducing the continuity requirement while retaining the bending physics. For the prescribed zero support displacements, define the trial and test spaces

$$\mathcal{S} = \{v \in H^2(0, 8) : v(0) = v(4) = v(8) = 0\}, \quad (3)$$

$$\mathcal{V} = \{\eta \in H^2(0, 8) : \eta(0) = \eta(4) = \eta(8) = 0\}. \quad (4)$$

The test function has homogeneous essential conditions because it represents an admissible virtual displacement.

On one element, let $Q = EIv'' = -M$. Equation (1) is $Q'' = q$. Multiplying it by $\eta \in \mathcal{V}$ and integrating gives

$$\int_{\Omega_e} \eta Q'' dx = \int_{\Omega_e} \eta q dx. \quad (5)$$

The first integration by parts gives

$$\int_{\Omega_e} \eta Q'' dx = [\eta Q']_{x_e}^{x_{e+1}} - \int_{\Omega_e} \eta' Q' dx. \quad (6)$$

Integrating the remaining domain term once more gives

$$- \int_{\Omega_e} \eta' Q' dx = -[\eta' Q]_{x_e}^{x_{e+1}} + \int_{\Omega_e} \eta'' Q dx. \quad (7)$$

Combining these results, using $Q = -M$ and $Q' = -V$, and moving the boundary contribution to the right produces

$$\boxed{\int_{\Omega_e} EI v'' \eta'' dx = \int_{\Omega_e} q \eta dx + [V \eta - M \eta']_{x_e}^{x_{e+1}}.} \quad (8)$$

- **Boundary Cancellations:** Internal terms (V , M) cancel at shared nodes and vanish at the $x = 0$ and $x = 8$ supports.
- **Load & Reaction Handling:** Point loads are incorporated during global assembly; support reactions are recovered after solving.
- **C^1 Requirement:** The presence of second derivatives (v'' , η'') in the weak form mandates C^1 continuity.

4 Elemental stiffness matrix and force vector

4.1 Two-node cubic Hermite interpolation

For an element $e = [x_1, x_2]$ of length L_e , define

$$r = \frac{x - x_1}{L_e}, \quad \mathbf{d}_e = [v_1 \quad \theta_1 \quad v_2 \quad \theta_2]^T, \quad \theta = v_{,x}.$$

A cubic interpolation is written as

$$v_h(r) = a_0 + a_1 r + a_2 r^2 + a_3 r^3.$$

The four coefficients are determined from the nodal displacement and physical-rotation conditions

$$v_h(0) = v_1, \quad v_{h,x}(0) = \theta_1, \quad v_h(1) = v_2, \quad v_{h,x}(1) = \theta_2.$$

Since

$$v_{h,x} = \frac{1}{L_e} \frac{dv_h}{dr},$$

these conditions give

$$a_0 = v_1, \quad a_1 = L_e \theta_1,$$

and

$$a_2 = -3v_1 - 2L_e \theta_1 + 3v_2 - L_e \theta_2,$$

$$a_3 = 2v_1 + L_e \theta_1 - 2v_2 + L_e \theta_2.$$

Substitution into the polynomial gives

$$v_h(r) = \mathbf{N}(r) \mathbf{d}_e,$$

where

$$N_1 = 1 - 3r^2 + 2r^3, \quad N_2 = L_e(r - 2r^2 + r^3), \quad N_3 = 3r^2 - 2r^3, \quad N_4 = L_e(-r^2 + r^3). \quad (9)$$

The first and third functions interpolate nodal displacements, while the second and fourth contain the factor L_e required to interpolate the physical nodal rotations.

4.2 Natural-coordinate transformation and Galerkin formulation

For numerical integration, introduce the natural coordinate $\xi \in [-1, 1]$ through

$$x = x_1 + \frac{L_e}{2}(1 + \xi).$$

The Jacobian is therefore

$$J = \frac{dx}{d\xi} = \frac{L_e}{2}, \quad \frac{d}{dx} = \frac{1}{J} \frac{d}{d\xi}, \quad dx = J d\xi.$$

Because the rotational degrees of freedom are physical slopes $\theta = v_{,x}$, the shape functions associated with rotations must be scaled by J when expressed in the natural coordinate:

$$N_{2i-1} = \hat{N}_{2i-1}, \quad N_{2i} = J \hat{N}_{2i}.$$

Thus,

$$v_h(\xi) = \mathbf{N}(\xi) \mathbf{d}_e, \quad v_{xx} = \frac{1}{J^2} \mathbf{N}_{\xi\xi} \mathbf{d}_e.$$

For the element weak form, choosing the Galerkin test function $\eta = N_i$ gives

$$\int_{\Omega_e} EI N_{i,xx} v_{h,xx} dx = \int_{\Omega_e} N_i q dx.$$

Substituting

$$v_{h,xx} = \sum_{j=1}^4 N_{j,xx} d_j$$

gives

$$\sum_{j=1}^4 \left(\int_{\Omega_e} EI N_{i,xx} N_{j,xx} dx \right) d_j = \int_{\Omega_e} N_i q dx.$$

Hence, the elemental stiffness and distributed-load entries are

$$k_{ij}^e = \int_{\Omega_e} EI N_{i,xx} N_{j,xx} dx, \quad f_i^{e,(q)} = \int_{\Omega_e} N_i q dx.$$

Using the natural-coordinate transformation,

$$N_{i,xx} = \frac{1}{J^2} N_{i,\xi\xi}, \quad dx = J \, d\xi,$$

so the elemental matrices can be evaluated as

$$\mathbf{k}_e = \frac{1}{J^3} \int_{-1}^1 EI(x(\xi)) \mathbf{N}_{,\xi\xi}^T \mathbf{N}_{,\xi\xi} \, d\xi,$$

and

$$\mathbf{f}_e^{(q)} = J \int_{-1}^1 \mathbf{N}^T q(x(\xi)) \, d\xi.$$

For constant EI and uniform q , direct evaluation of these integrals gives the standard cubic Hermite element matrices:

$$\mathbf{k}_e = \frac{EI}{L_e^3} \begin{bmatrix} 12 & 6L_e & -12 & 6L_e \\ 6L_e & 4L_e^2 & -6L_e & 2L_e^2 \\ -12 & -6L_e & 12 & -6L_e \\ 6L_e & 2L_e^2 & -6L_e & 4L_e^2 \end{bmatrix} \quad (10)$$

$$\mathbf{f}_e^{(q)} = \frac{qL_e}{2} [1 \quad L_e/6 \quad 1 \quad -L_e/6]^T \quad (11)$$

Concentrated forces and moments are assembled separately at their corresponding mesh nodes.

5 Finite-element code and implementation

The program is divided into general modules for input, polynomial operations, shape-function generation, quadrature, element matrices, preprocessing, assembly, solution, and post-processing. The program is fully input-driven, with all problem-specific parameters defined in `input.jl`.

5.1 Polynomial representation and generalized Hermite interpolation

The custom `Poly` type stores polynomial coefficients in ascending power order and supports arithmetic, evaluation, and exact differentiation. This allows generated shape functions to be differentiated directly for rotation and curvature.

For n interpolation nodes in $\xi \in [-1, 1]$, displacement and slope conditions provide $2n$ interpolation constraints. Hence the minimum complete polynomial degree is

$$p = 2n - 1.$$

With

$$\mathbf{p}(\xi) = [1 \quad \xi \quad \xi^2 \quad \dots \quad \xi^{2n-1}]^T,$$

write

$$v_h(\xi) = \mathbf{p}(\xi)^T \mathbf{a}.$$

Collecting the displacement and slope conditions at all nodes gives

$$\boxed{\mathbf{C}\mathbf{a} = \mathbf{g}}, \quad C_{2i-1,j} = p_j(\xi_i), \quad C_{2i,j} = p'_j(\xi_i).$$

Thus the generalized parent-coordinate basis is

$$\boxed{\widehat{\mathbf{N}}(\xi) = \mathbf{C}^{-T} \mathbf{p}(\xi)}, \quad v_h(\xi) = \widehat{\mathbf{N}}(\xi)^T \mathbf{g}.$$

The odd functions interpolate nodal displacement and the even functions interpolate nodal parent-coordinate slopes.

5.2 Gauss–Legendre quadrature

Gauss–Legendre quadrature approximates an integral over $[-1, 1]$ as

$$\int_{-1}^1 f(\xi) d\xi \approx \sum_{g=1}^{n_g} w_g f(\xi_g), \quad (12)$$

where ξ_g are the quadrature points and w_g are their corresponding weights. For an n_g -point rule, the points are chosen as the roots of the Legendre polynomial $P_{n_g}(\xi)$:

$$P_{n_g}(\xi_g) = 0. \quad (13)$$

This choice makes the quadrature rule exact for polynomials of degree up to $2n_g - 1$.

The Legendre polynomials are generated using

$$(k+1)P_{k+1}(\xi) = (2k+1)\xi P_k(\xi) - kP_{k-1}(\xi), \quad (14)$$

with $P_0 = 1$ and $P_1 = \xi$.

To obtain the roots numerically, the polynomial is converted to a companion matrix. For

$$p(\xi) = \xi^n + c_{n-1}\xi^{n-1} + \cdots + c_0, \quad (15)$$

the companion matrix is constructed by keeping 1's on the subdiagonal and placing the polynomial coefficients in the last column with a negative sign:

$$C = \begin{bmatrix} 0 & 0 & \cdots & 0 & -c_0 \\ 1 & 0 & \cdots & 0 & -c_1 \\ 0 & 1 & \cdots & 0 & -c_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & -c_{n-1} \end{bmatrix}. \quad (16)$$

This matrix satisfies

$$\det(\lambda I - C) = p(\lambda), \quad (17)$$

so its eigenvalues are the roots of $p(\xi)$. Hence, for $P_{n_g}(\xi)$, the eigenvalues of its companion matrix give the Gauss points. Finally, the weights are calculated from

$$w_g = \frac{2}{(1 - \xi_g^2)[P'_{n_g}(\xi_g)]^2}. \quad (18)$$

Thus, the implemented procedure is $P_{n_g}(\xi) \rightarrow C \rightarrow \{\xi_g\} \rightarrow \{w_g\}$, which provides the points and weights required for numerical integration.

5.3 Automatic mesh generation and LM mapping

The preprocessor combines beam ends with all support, load, moment, and release coordinates, creating nodes at each and distributing remaining elements between them. Thus, h -refinement preserves all discontinuity and constraint locations.

The location matrix $\text{LM}(a, e)$ maps local DOFs to global equation numbers, with 2nnpe rows, supporting both cubic and higher-order elements. Distributed loads are assembled normally, while point forces and moments are applied directly to their corresponding nodal DOFs. Hence, the node at $x = 6\text{ m}$ allows the 120 kN force to be applied exactly, without entering a distributed-load integral.

5.4 Internal releases through DOF topology

Releases are represented by changing the LM connectivity rather than by post-hoc modification of the element stiffness:

Input	Shared DOF	Split DOF	Released resultant
:m	displacement v	rotation θ	moment M
:v	rotation θ	displacement v	shear V

For a moment release, the adjacent elements therefore retain a common deflection but receive independent rotation equation numbers. For a shear release, the converse is used. This directly expresses the permitted kinematic discontinuity while retaining the required compatible DOF.

5.5 Constraints, solution, and reactions

Prescribed displacements or rotations are applied by setting the corresponding row of the stiffness matrix to zero, except for the diagonal entry, which is set to 1. The corresponding entry in the force vector is set to the prescribed value. The resulting system is then solved using Julia's backslash operator. The original stiffness matrix and force vector are retained to calculate the reactions, allowing reactions to be recovered as

$$\boxed{R = K_0 d - F_0.} \quad (19)$$

5.6 Post-processing and interpolated extrema

Within every element,

$$v_h(\xi) = \sum_{a=1}^{2n} N_a(\xi) d_a, \quad \theta_h = \frac{dv_h}{dx}.$$

The maximum deflection is found by sampling this interpolated field throughout every element, rather than selecting only the largest nodal displacement. Shear and bending-moment diagrams are reconstructed from loading and recovered reactions, including their jumps at concentrated actions.

6 Deflection, shear-force, and bending-moment profiles

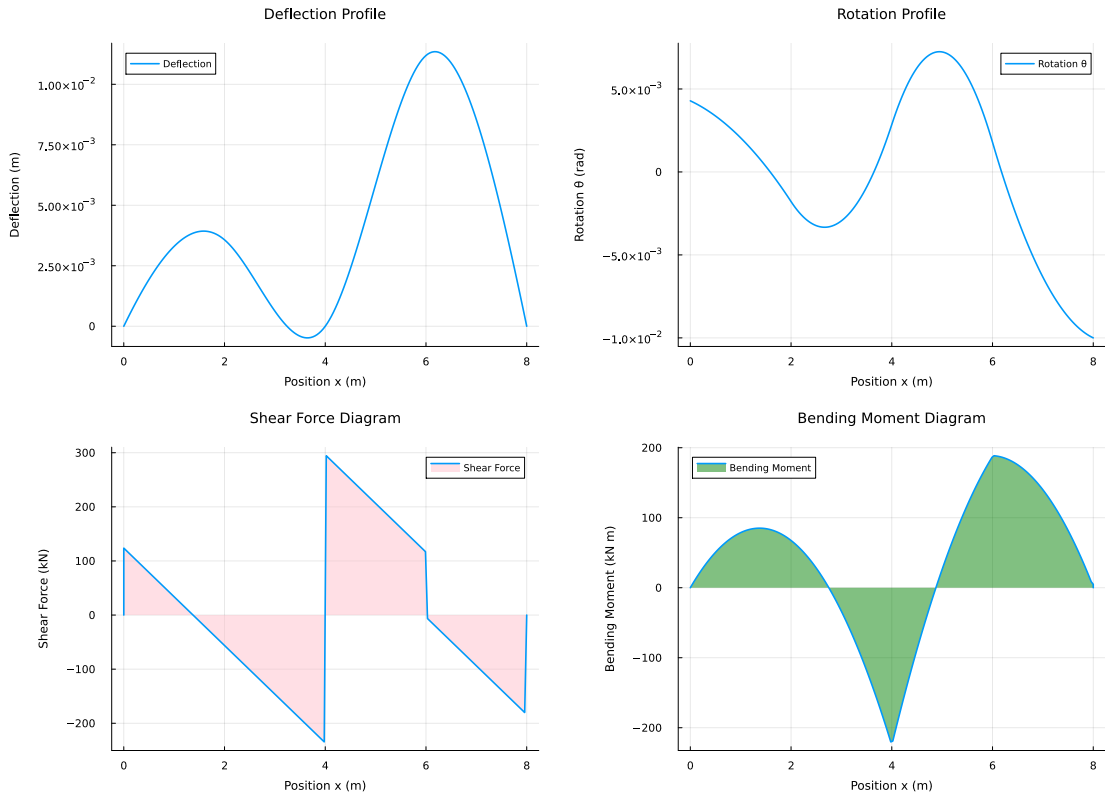


Figure 2: Computed deflection, rotation, shear-force and bending-moment responses for the selected mesh.

The vertical reactions must satisfy

$$R_0 + R_4 + R_8 = 840 \text{ kN} = qL + P, \quad (20)$$

together with moment equilibrium about either beam end. Using the displayed values, both the vertical-force residual and the moment residual about $x = 0$ are zero to the reported precision. The shear diagram jumps at each support and at x_P , while the bending-moment diagram remains continuous across the point force and vanishes at the simple end supports.

Table 1: Principal numerical results.

Quantity	Value	Position
Maximum absolute deflection	11.348 mm	6.180 m
Reaction at $x = 0$	123.750 kN	0 m
Reaction at $x = 4$	532.500 kN	4 m
Reaction at $x = 8$	183.750 kN	8 m
Maximum $ V $	294.441 kN	4.020 m
Maximum $ M $	220.269 kN m	3.979 m

7 Maximum deflection versus total number of elements

7.1 h -refinement procedure

To evaluate the model's convergence, a h -refinement study is performed. The interpolation order is held constant using the standard two-node cubic Hermite beam element, while the total number of elements is systematically doubled to halve the characteristic length, h .

During this sequence, mandatory nodes must be strictly preserved at all physical supports and applied point loads. This guarantees that boundary conditions are perfectly captured and loads are not artificially smeared across the elements.

To determine when the mesh is sufficiently dense, we track the maximum vertical deflection, $w_{\max}$, across each iteration. Convergence is evaluated by calculating the successive relative change, ε_i , which measures the percentage difference between the current mesh (i) and the previous, less refined iteration ($i - 1$):

$$\varepsilon_i = \frac{|w_{\max}^{(i)} - w_{\max}^{(i-1)}|}{|w_{\max}^{(i)}|} \times 100\% \quad (21)$$

As the mesh is refined, this relative change steadily decreases until it falls below a set tolerance threshold, indicating a converged solution.

8 Maximum deflection versus higher-order approximation

8.1 p -refinement procedure

The mandatory element partition is held fixed while `nnpe` is increased from 2 to 10. The corresponding polynomial degree is $p = 2 \text{nnpe} - 1$. This compares increasingly rich C^1 Hermite spaces without changing the principal element boundaries. As the number of nodes per element increases, the monomial interpolation matrix $\mathbf{C}$ becomes larger and increasingly ill-conditioned. This raises the cost of constructing the basis and eventually amplifies round-off error, explaining the loss of accuracy visible at the highest tested orders.

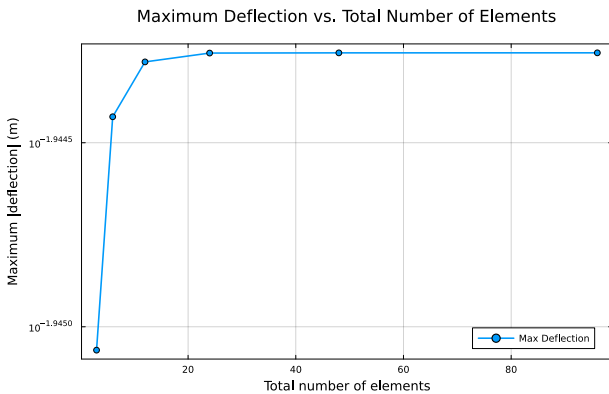
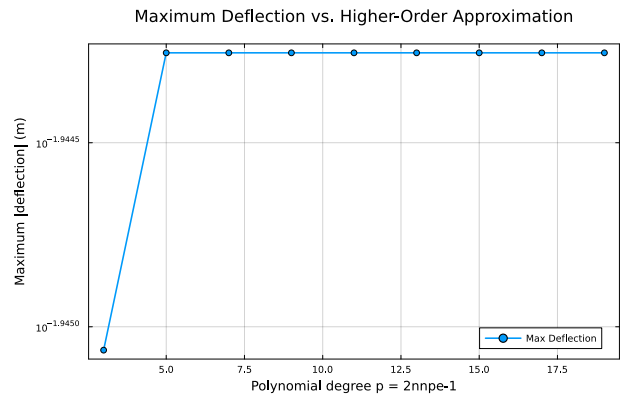
(a) h -refinement.(b) p -refinement.

Figure 3: Change in interpolated maximum deflection under refinement.

Conclusions

The modular implementation reproduced the analytical cubic element matrices and solved the assigned continuous beam using input-defined geometry, properties, supports and loads. Automatic special-node insertion, LM-based release handling, polynomially generated Hermite bases and generalized Gauss–Legendre quadrature make the formulation reusable beyond the assigned case. The maximum computed deflection is 11.348 mm at $x = 6.180$ m. The reactions satisfy vertical-force and moment equilibrium to the reported precision. The h -refinement study approaches a stable response, whereas very high polynomial orders become sensitive to the conditioning of the monomial interpolation system.

References

1. ME3180 course notes, *Finite Element Method Theory*, 2026.
2. J. P. Boyd, *Solving Transcendental Equations: The Chebyshev Polynomial Proxy and Other Numerical Rootfinders, Perturbation Series, and Oracles*. Philadelphia: Society for Industrial and Applied Mathematics, 2014.
3. Julia documentation, <https://docs.julialang.org/>.

Group contributions

Table 2: Group members and contributions.

Name	Roll No.	Function(s)/type written	Contributions
Devadath Sabarish	CO24BTECH11006	<code>Poly</code> , <code>ShapeFunctions</code> , <code>Report</code>	Polynomial operator framework, generalized Hermite shape-function generator, and the report structure
Guddeti Sreeteja	CO24BTECH11009	<code>kelem</code> , <code>felem</code> , <code>sampled_max_deflection</code>	Element stiffness/load routines; corrected interpolated maximum and h/p refinement
Himesh Katru	CO24BTECH11010	<code>legendre_polynomial</code> , <code>gauss_quadrature</code> , <code>global_kf</code>	Generated Gauss–Legendre rule and global stiffness/load assembly
Mannepula Rishith	CO24BTECH11014	<code>postprocess</code> , <code>plot_results</code>	Deflection/rotation and shear/moment recovery, plotting, and reaction-related postprocessing
Tanish Verma	CO24BTECH11023	<code>generateMesh</code> , <code>generateLM</code> , <code>Solve</code> , <code>Report</code>	Input model, special-node mesh, release-aware LM mapping, constraints, solver and initial convergence driver